Canteiro C3 / FURG

Material de estudo para a apresentação: lógica do trabalho, matrizes e comandos OpenGL

Visão geral

O trabalho reproduz em 3D, com OpenGL, o canteiro C3 da FURG: uma relva com contorno ondulado (curva polar `r = 3 + cos(3θ)`) cercada por tijolos, com um poste de luz e uma placa sobre um suporte cilíndrico. A cena é interativa: o usuário gira, inclina, move e dá zoom na câmera pelo teclado. Foi feito em Python com PyOpenGL e GLFW, implementando à mão as matrizes e os algoritmos principais.

Controles

- Setas: giram e inclinam a cena | W A S D: movem o ponto de vista | + / -: zoom
- P: projeção perspectiva | O: projeção ortogonal
- B: liga/desliga as linhas de Bresenham | H: liga/desliga a sombra do sol

1. Lógica do trabalho

Uma cena 3D é redesenhada a cada frame. O **main.py** controla a janela, a câmera e a ordem de desenho; os demais arquivos fornecem as matrizes, as formas, a iluminação e a rasterização.

main.py (o maestro)

- **Janela e laço principal:** cria a janela com GLFW (pedindo buffer de stencil de 8 bits) e repete `poll_events → renderizar → swap_buffers` até fechar.
- **Estado da câmera:** variáveis globais guardam rotação (`rotacao_x`, `rotacao_y`), zoom (`distancia_cam`), deslocamento (`trans_x`, `trans_y`) e os interruptores `usar_perspectiva`, `mostrar_rasterizacao` e `mostrar_sombra`.
- `tratar_teclado`**:** cada tecla altera uma dessas variáveis.
- `aplicar_projecao`**:** carrega a matriz de perspectiva ou ortogonal em `GL_PROJECTION` com `glMultMatrixf`.
- `direcao_sol_no_mundo`**:** o sol é fixo na tela, enquanto a cena gira. A função desfaz as rotações da cena para descobrir a direção do sol no espaço do mundo; é isso que faz a sombra mudar de direção quando se gira a cena.
- `renderizar`**:** monta o frame na ordem abaixo.

```
1. limpa cor, profundidade e stencil
2. aplica a projeção
3. posiciona o sol (modelview ainda sem a câmera)
4. aplica a câmera (translação + rotações por matrizes)
5. desenha eixos e relva (a relva marca o stencil com 1)
6. desenha as sombras
7. desenha tijolos, poste e placa
8. desenha as linhas de Bresenham, se ativadas
```

matrizes.py (a matemática)

Funções que devolvem matrizes 4x4 com NumPy: translação, rotação em X e em Y, escala, perspectiva, ortogonal e **sombra plana**. As matrizes são escritas por linhas (como nos livros), por isso o código passa `.T` (transposta) ao `glMultMatrixf`, que espera a ordem por colunas.

A matriz de sombra projeta qualquer ponto sobre o plano do chão (y = 0) ao longo da direção da luz.

geometria.py (o ambiente)

- `raio_canteiro(θ)`**:** a curva do contorno, 3 + cos(3θ).
- `desenhar_relva`**:** um leque de triângulos (`GL_TRIANGLE_FAN`) do centro até o contorno, com normal para cima.
- `desenhar_borda_tijolos`**:** coloca 120 tijolos ao longo da curva; o ângulo de cada um vem da tangente, calculada pela diferença entre pontos vizinhos.
- `desenhar_tijolo`**:** um cubo unitário, dimensionado por `matriz_escala`, girado por `matriz_rotacao_y` e posicionado por `matriz_translacao`.
- `desenhar_cilindro` **e** `desenhar_esfera`**:** formas básicas montadas com quadriláteros e normais.
- `desenhar_poste` **e** `desenhar_placa`**:** compõem as formas básicas com `glPushMatrix/glPopMatrix` e matrizes próprias.
- `desenhar_eixos`**:** eixos X, Y e Z coloridos, como referência.
- `desenhar_sombras`**:** redesenha tijolos, poste e placa achatados no chão pela matriz de sombra, em preto translúcido. O stencil restringe a sombra à relva e impede que ela escureça em dobro. O helper `cor(...)` ignora as cores originais enquanto a flag `modo_sombra` está ativa, para a sombra sair preta.

iluminacao.py (luz e visibilidade)

- `inicializar_iluminacao`**:** liga o Z-buffer (`GL_DEPTH_TEST`), a remoção de faces traseiras (`GL_CULL_FACE`), a luz `GL_LIGHT0` e o `GL_SMOOTH` (Gouraud). Define componentes ambiente, difuso e especular, e o material com brilho (Phong).
- `POSICAO_SOL` **e** `posicionar_sol`**:** o sol é uma luz direcional (w = 0), reenviada a cada frame para ficar fixa em relação à tela.

rasterizacao.py (Bresenham)

Converte os extremos da linha para uma grade inteira (escala 50) e avança ponto a ponto usando só somas e comparações, sem divisão nem arredondamento. Cada ponto é desenhado com `GL_POINTS` em amarelo, um pouco acima do chão. O main.py o usa para traçar 6 linhas do centro até a borda do canteiro.

Arquivos de apoio

Arquivo	Função
comandos.txt	Passo a passo para clonar, criar o ambiente virtual, instalar e executar, além da lista de controles.
requirements.txt	Dependências fixas: PyOpenGL, glfw e numpy.
README.md	Apresentação do projeto.
LICENSE	Licença do repositório.
_gitignore	Deve se chamar **.gitignore**. Diz ao Git quais arquivos não enviar (venv/, __pycache__/).

Resumo do fluxo de um frame

```
teclado → altera variáveis → renderizar():
   projeção (matrizes.py) → sol (iluminacao.py) → câmera (matrizes.py)
   → relva + stencil → sombras → tijolos/poste/placa (geometria.py)
   → Bresenham (rasterizacao.py)
```

2. Como funcionam as matrizes

Por que matrizes 4x4?

Cada ponto 3D (x, y, z) é escrito como (x, y, z, 1): as chamadas **coordenadas homogêneas**. Com isso, translação, rotação, escala e projeção viram a mesma operação: multiplicar uma matriz 4x4 pelo ponto. Uma matriz 3x3 não conseguiria fazer translação; por isso existe a quarta linha e coluna.

```
ponto_novo = Matriz × ponto
```

As matrizes do matrizes.py

Translação: a identidade com o deslocamento na última coluna.

```
[1 0 0 tx]   [x]   [x + tx]
[0 1 0 ty] × [y] = [y + ty]
[0 0 1 tz]   [z]   [z + tz]
[0 0 0 1 ]   [1]   [  1   ]
```

Exemplo: o ponto (1, 0, 0) com tx = 2 e ty = 3 vira (3, 3, 0).

Escala: uma diagonal com os fatores. `matriz_escala(0.25, 0.15, 0.4)` transforma o cubo unitário do tijolo em um bloco de 0,25 × 0,15 × 0,4.

Rotação em Y: o eixo Y não muda, e X e Z giram entre si. A rotação em X é análoga, mudando Y e Z. O ângulo vai em radianos (por isso o código usa `np.radians`).

```
x' =  cos·x + sin·z
z' = -sin·x + cos·z
```

Perspectiva: aqui está a "mágica" do 3D.

- `f = 1 / tan(fovy/2)` controla o campo de visão, e `aspect` corrige a proporção da janela.
- A última linha `[0, 0, -1, 0]` copia -z para a coordenada w.
- Depois, o OpenGL divide x, y e z por w. Quanto mais longe o ponto (maior -z), maior o w e menor ele aparece: objetos distantes encolhem.
- Os termos da terceira linha comprimem a profundidade entre `z_near` e `z_far` no intervalo que o Z-buffer usa.

Ortogonal: mapeia a caixa [left, right] × [bottom, top] × [near, far] para o cubo [-1, 1]. A última linha é `[0, 0, 0, 1]`, então w continua 1: não há divisão e objetos distantes não encolhem.

Sombra plana: a matriz é (plano·L)·I − L⊗plano. Ela projeta cada ponto sobre o chão (y = 0) seguindo a direção da luz. Na prática, zera a altura e desloca x e z conforme a direção do sol, achatando o objeto no solo.

O `.T` (transposta)

O NumPy guarda a matriz **por linhas**, mas o glMultMatrixf lê **por colunas**. Sem o .T, a matriz seria interpretada trocada (a translação, por exemplo, iria para a linha de baixo e nada funcionaria). Por isso toda chamada é `glMultMatrixf(m.T)`.

A ordem importa

O glMultMatrixf multiplica a matriz atual **pela direita** (`atual = atual × nova`). Na prática, a **última** matriz aplicada é a **primeira** a agir sobre o ponto. No desenhar_tijolo:

```
glMultMatrixf(T)   # 3º a agir: posiciona na borda
glMultMatrixf(R)   # 2º: gira para seguir a curva
glMultMatrixf(S)   # 1º: dá o tamanho ao cubo unitário
```

Lendo de baixo para cima: o cubo é dimensionado, depois girado e por fim posicionado. Trocar a ordem muda o resultado (girar depois de posicionar faria o tijolo orbitar a origem). O mesmo vale para a câmera no renderizar: Translação · Rotação X · Rotação Y.

As duas matrizes principais do OpenGL

- `GL_PROJECTION`**:** recebe perspectiva ou ortogonal (`aplicar_projecao`).
- `GL_MODELVIEW`**:** recebe a câmera e as transformações dos objetos.

O `glPushMatrix` salva a modelview atual e o `glPopMatrix` a restaura. É assim que cada tijolo, o poste e a placa têm sua própria posição sem afetar os outros: salva, aplica as transformações do objeto, desenha e volta ao estado anterior.

O caminho completo de um vértice

```
vértice (x, y, z, 1)
   → matrizes dos objetos (T·R·S)      ┐
   → matriz da câmera (T · Rx · Ry)    ┘ MODELVIEW
   → matriz de projeção                  PROJECTION
   → divisão por w                       (perspectiva)
   → viewport                            posição final na tela
```

Ressalva sobre a sombra

Para o chão receber a sombra, a matriz de sombra é multiplicada **logo depois da câmera e antes das matrizes dos objetos**. Ela precisa agir no espaço do mundo, depois de cada objeto estar na sua posição e antes de a câmera mover tudo. Por isso o desenhar_sombras é chamado quando a modelview contém só a câmera.

3. Comandos OpenGL usados no projeto

Configuração e estado

Comando	Para que serve
`glClearColor(r,g,b,a)`	Define a cor de fundo (o céu azul) usada ao limpar a tela.
`glClear(...)`	Limpa os buffers no início de cada frame: cor, profundidade (GL_DEPTH_BUFFER_BIT) e stencil (GL_STENCIL_BUFFER_BIT).
`glEnable(x) / glDisable(x)`	Liga ou desliga um recurso (iluminação, depth test, culling, blend, stencil etc.).
`glViewport(x,y,w,h)`	Define a área da janela onde a cena é desenhada. Atualizado quando a janela é redimensionada.

Visibilidade

Comando	Para que serve
`GL_DEPTH_TEST`	Liga o **Z-buffer**: de dois pontos no mesmo pixel, aparece o mais próximo.
`glDepthFunc(GL_LESS)`	Regra do Z-buffer: passa o fragmento que estiver mais perto.
`glDepthMask(GL_FALSE/TRUE)`	Permite ou impede a escrita no buffer de profundidade. Usado na sombra, para ela não bloquear outros objetos.
`GL_CULL_FACE` e `glCullFace(GL_BACK)`	Descarta faces de trás dos polígonos (**back-face culling**), economizando desenho.

Matrizes e transformações

Comando	Para que serve
`glMatrixMode(GL_PROJECTION / GL_MODELVIEW)`	Escolhe qual matriz será alterada: a de projeção ou a de modelo/câmera.
`glLoadIdentity()`	Zera a matriz atual (volta à identidade).
`glMultMatrixf(m)`	Multiplica a matriz atual por uma matriz 4x4 nossa (translação, rotação, escala, projeção, sombra).
`glPushMatrix()` / `glPopMatrix()`	Salva e restaura a matriz atual. Cada objeto tem suas transformações sem afetar os demais.

Iluminação e material

Comando	Para que serve
`GL_LIGHTING` e `GL_LIGHT0`	Ligam o sistema de iluminação e a luz número 0 (o sol).
`glLightfv(GL_LIGHT0, ...)`	Define propriedades da luz: posição/direção (GL_POSITION), ambiente, difusa e especular.
`GL_COLOR_MATERIAL` e `glColorMaterial(...)`	Faz o glColor definir a cor ambiente e difusa do material.
`glMaterialfv(..., GL_SPECULAR, ...)` e `glMaterialf(..., GL_SHININESS, ...)`	Definem o brilho do material (reflexo especular do Phong).
`glShadeModel(GL_SMOOTH)`	**Tonalização de Gouraud**: interpola as cores entre os vértices.
`GL_NORMALIZE`	Renormaliza as normais; necessário quando há escala.

Comando	Para que serve
`glNormal3f(x,y,z)`	Define a normal da superfície, usada no cálculo da iluminação.

Desenho de geometria

Comando	Para que serve
`glBegin(modo) / glEnd()`	Delimitam uma sequência de vértices desenhados com o mesmo tipo de primitiva.
`GL_QUADS`	Quadriláteros (faces do tijolo, painel e cilindro).
`GL_TRIANGLE_FAN`	Leque de triângulos (relva e tampa do cilindro).
`GL_QUAD_STRIP`	Faixa de quadriláteros (esfera).
`GL_LINES`	Linhas (eixos X, Y e Z).
`GL_POINTS`	Pontos (pixels do Bresenham).
`glVertex3f(x,y,z)`	Define um vértice.
`glColor3f(r,g,b)`	Define a cor atual (RGB).
`glColor4f(r,g,b,a)`	Define a cor com transparência (sombra preta a 45%).
`glLineWidth(w)`	Espessura das linhas dos eixos.
`glPointSize(s)`	Tamanho dos pontos do Bresenham.

Sombra (blend, polygon offset e stencil)

Comando	Para que serve
`GL_BLEND` e `glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA)`	Mistura a cor da sombra com o que já está desenhado, dando a translucidez.
`GL_POLYGON_OFFSET_FILL` e `glPolygonOffset(f,u)`	Desloca levemente a profundidade da sombra para evitar *z-fighting* com a relva.
`GL_STENCIL_TEST`	Liga o buffer de stencil, uma máscara por pixel.
`glStencilFunc(func, ref, mask)`	Define a condição de teste do stencil (por exemplo, passar só onde vale 1).
`glStencilOp(sfail, dpfail, dppass)`	Define o que fazer com o valor do stencil conforme o resultado dos testes. A relva grava 1 e a sombra zera ao desenhar.

Comandos só de teste ou substituídos

- `glRotatef`: substituído por `glMultMatrixf` com `matriz_rotacao_x/y`, para cumprir o requisito das matrizes.
- `glGetError` e `glGetIntegerv(GL_STENCIL_BITS)`: usados só para diagnóstico.

Comandos da biblioteca GLFW (janela e entrada)

Comando	Para que serve
`glfw.init() / glfw.terminate()`	Inicia e encerra a biblioteca.
`glfw.window_hint(...)`	Configura a janela antes de criá-la (pedido de 8 bits de stencil).
`glfw.create_window(...)`	Cria a janela com o contexto OpenGL.

Comando	Para que serve
`glfw.make_context_current(...)`	Torna esse contexto o ativo para os comandos GL.
`glfw.set_key_callback(...)` e `set_window_size_callback(...)`	Registram as funções chamadas ao pressionar teclas e ao redimensionar.
`glfw.poll_events()`	Processa os eventos pendentes (teclado, janela).
`glfw.swap_buffers(...)`	Mostra o frame pronto na tela (buffer duplo).
`glfw.window_should_close(...)`	Indica se o usuário fechou a janela, encerrando o laço principal.